Web Application Development – Lecture 9: React Router

SWE 381 – Web Application Development

Dr. Ohoud Alharbi Dr. Achraf Gazdar

Department of Software Engineering
College of Computer and Information Sciences
King Saud University

Term 2 – 2025/2026

Learning Objectives

By the end of this lecture, you will be able to:

- Explain the difference between **server-side** and **client-side** routing
- Set up **React Router v6** with `BrowserRouter`, `Routes`, and `Route`
- Build navigation menus with `Link` and `NavLink` (active styling)
- Navigate programmatically with the `useNavigate` hook
- Access **URL parameters** using the `useParams` hook
- Implement a **ProtectedRoute** component for authenticated pages
- Design a **full project route map** for a real application

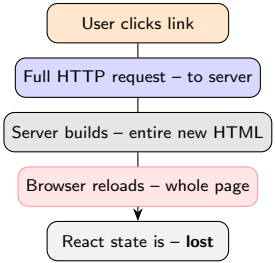
Agenda

1. Server vs Client-Side Routing
2. BrowserRouter, Routes, Route
3. Link vs NavLink
4. useNavigate
5. useParams
6. ProtectedRoute Component
7. Full Project Route Map
8. Recap

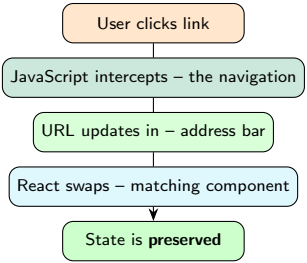
Outline

Server-Side vs Client-Side Routing

Server-Side Routing (traditional):



Client-Side Routing (React Router):



Why Client-Side Routing?

Feature	Server-side	Client-side
Page reload	Full reload	No reload
Network request	Every page	First load only
Speed (after load)	Slower	Instant
State preservation	Lost on nav	Preserved
Smooth transitions	No	Yes
Bookmarkable URLs	Yes	Yes
SEO (default)	Easy	Needs setup

React Router – W3Schools

React Router is a library that lets you handle **client-side routing** in React. It **maps URL paths to components**, enabling navigation between pages without refreshing the browser.

Install React Router:

```
npm install react-router-dom
```

Core components (v6):

- `BrowserRouter` – wraps the whole app
- `Routes` – container for all routes
- `Route` – maps a path to a component
- `Link` – navigation without reload
- `NavLink` – Link with active styling
- `useNavigate` – programmatic navigation
- `useParams` – read URL parameters
- `Navigate` – redirect component

Outline

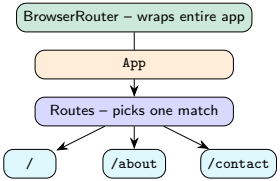
Setting Up React Router

W3Schools – React Router

To use React Router, wrap your app with `BrowserRouter`. Then use `Routes` as a container and `Route` components to map paths to components.

main.jsx – one-time setup:

```
import { createRoot } from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App';
createRoot(document.getElementById('root'))
  .render(
    <BrowserRouter>
      <App />
    </BrowserRouter>
  );
```



App.jsx – define routes:

```
import { Routes, Route } from 'react-router-dom';
import Home    from './pages/Home';
import About   from './pages/About';
import Contact from './pages/Contact';
function App() {
  return (
    <Routes>
      <Route path="/"        element={<Home />}    />
      <Route path="/about"   element={<About />}   />
      <Route path="/contact" element={<Contact />} />
    </Routes>
```

Route element prop

In React Router v6, `Route` uses an `element` prop with JSX. No more `component={...}`.

Route – Path Matching Rules

```jsx
<Routes>
  {/* Exact root -- only matches "/" */}
  <Route path="/"        element={<Home />}     />

  {/* Matches "/about" exactly */}
  <Route path="/about"   element={<About />}    />

  {/* Dynamic segment -- :id is a parameter */}
  <Route path="/courses/:id"
                         element={<CourseDetail />} />

  {/* Nested child routes */}
  <Route path="/admin"   element={<AdminLayout />}>
    <Route path="users"  element={<Users />}    />
    <Route path="stats"  element={<Stats />}    />
  </Route>

  {/* Catch-all -- show 404 for unknown paths */}
  <Route path="*"        element={<NotFound />} />
</Routes>
```

URL visited	Renders
/	Home
/about	About
/courses/3	CourseDetail
/admin/users	AdminLayout + Users
/admin/stats	AdminLayout + Stats
/oops	NotFound

v6 vs v5

In v6, Routes automatically picks the **best match**. No exact prop needed – that was a v5 requirement.

Catch-all route

Always put path="*" **last**. It matches anything not matched above – a perfect 404 page.

W3Schools Complete Router Example

```jsx
import { BrowserRouter, Routes, Route } from 'react-router-dom';

function Home()     { return <h1>Home Page</h1>;     }
function About()    { return <h1>About Page</h1>;    }
function Contact() { return <h1>Contact Page</h1>; }
function NotFound(){ return <h1>404 - Page Not Found</h1>; }

function App() {
  return (
    <BrowserRouter>
      {/* Nav and Routes live inside BrowserRouter */}
      <nav>
        <a href="/">Home</a> |{" "}
        <a href="/about">About</a> |{" "}
        <a href="/contact">Contact</a>
        {/* NOTE: using <a> here causes page reload!
            We will replace these with <Link> next. */}
      </nav>

      <Routes>
        <Route path="/"        element={<Home />}     />
        <Route path="/about"   element={<About />}    />
        <Route path="/contact" element={<Contact />}  />
        <Route path="*"        element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}
```

⚠ Never use `<a href>` for internal navigation

A plain HTML `<a>` tag triggers a full page reload, destroying React state. Use `<Link>` from `react-router-dom` instead.

Outline

Link – Basic Navigation Without Reload

W3Schools – React Router

React Router provides the `Link` component to create navigation links. Unlike an HTML `<a>` tag, `Link` prevents the full page reload and lets React Router handle the navigation.

```jsx
import { Link } from 'react-router-dom';

function Navbar() {
  return (
    <nav>
      {/* Link uses the "to" prop -- no href */}
      <Link to="/">Home</Link>  |{" "}
      <Link to="/about">About</Link>  |{" "}
      <Link to="/contact">Contact</Link>

      {/* Link to a dynamic path */}
      <Link to="/courses/3">Course 3</Link>

      {/* Link with state */}
      <Link to="/profile"
        state={{ from: "navbar" }}>
        Profile
      </Link>
    </nav>
  );
}
```

`<a href>`	`<Link to>`
Full page reload	No reload
React state lost	State preserved
Server request	Client-side only
HTML standard	React Router

Rendered HTML

`<a href="/about">About</a>` – [2pt] Link renders as an `<a>` tag in the DOM, but React Router intercepts the click and handles it client-side.

NavLink – Active Styling

W3Schools – NavLink

`NavLink` is like `Link` but it knows whether the route is **currently active**. It provides an `isActive` flag that you can use to apply different styles to the current page's link.

```jsx
import { BrowserRouter, Routes, Route,
         NavLink } from 'react-router-dom';
// style function receives { isActive }
const navLinkStyles = ({ isActive }) => ({
  color:      isActive ? '#007bff' : '#333',
  textDecoration: isActive ? 'none':'underline',
  fontWeight: isActive ? 'bold'  : 'normal',
  padding: '5px 10px'
});
function App() {
  return (
    <BrowserRouter>
      <nav>
        <NavLink to="/" style={navLinkStyles}>
          Home</NavLink>
        <NavLink to="/about" style={navLinkStyles}>
          About</NavLink>
        <NavLink to="/contact" style={navLinkStyles}>
          Contact</NavLink>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact"
               element={<Contact />} />
      </Routes>
```

On Home page (/)

Home About Contact – [4pt] **Home Page**

On About page (/about)

Home **About** Contact – [4pt] **About Page**

className variant

You can also use `className` instead of `style`: – `className={({isActive}) => isActive ? "active" : ""}`

Link vs NavLink – Comparison

Feature	`Link`	`NavLink`
Prevents page reload	Yes	Yes
Active state detection	No	Yes
Style / className function	No	Yes – receives `isActive`
Automatic `.active` class	No	Yes (default)
Use case	Any internal link	Navigation menus
Import	`react-router-dom`	`react-router-dom`

When to use Link

Use `Link` for any internal link that doesn't need active styling – e.g. a "Back to list" button, a card that links to a detail page, or inline text links.

When to use NavLink

Use `NavLink` for navigation menus (top nav, sidebar) where you want the current page's link to appear **highlighted** or **bold**.

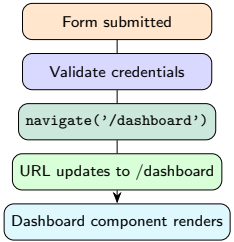
Outline

useNavigate – Programmatic Navigation

W3Schools – useNavigate

The `useNavigate` hook allows you to navigate the user to a new page **without the user clicking a link**. It is useful after form submissions, authentication checks, or timed redirects.

```jsx
import { useNavigate } from 'react-router-dom';

function LoginPage() {
  const navigate = useNavigate();
  const [name, setName] = useState("");

  const handleLogin = (e) => {
    e.preventDefault();
    // ... validate credentials ...

    // Navigate to dashboard after login
    navigate('/dashboard');
  };

  return (
    <form onSubmit={handleLogin}>
      <input value={name}
        onChange={e => setName(e.target.value)}
        placeholder="Enter name" />
      <button type="submit">Login</button>
    </form>
  );
}
```



useNavigate – Advanced Usage

```jsx
import { useNavigate } from 'react-router-dom';

function CourseDetail({ courseId }) {
  const navigate = useNavigate();

  // 1. Navigate to a path
  const goHome = () => navigate('/');

  // 2. Navigate with dynamic path
  const goToCourse = (id) =>
    navigate(`/courses/${id}`);

  // 3. Go BACK in history (-1 = previous page)
  const goBack = () => navigate(-1);

  // 4. Go FORWARD in history
  const goForward = () => navigate(1);

  // 5. Replace current history entry
  //    (no back button to return from)
  const redirect = () =>
    navigate('/login', { replace: true });

  return (
    <div>
      <button onClick={goBack}>Back</button>
      <button onClick={() => goToCourse(5)}>
        Go to Course 5
      </button>
      <button onClick={redirect}>
        Go to Login (no back)
      </button>
    </div>
```

Call	Effect
`navigate('/path')`	Go to /path
`navigate(-1)`	Browser Back
`navigate(1)`	Browser Forward
`navigate('/path', {replace:true})`	No history entry

Link vs useNavigate

Prefer `Link`/`NavLink` for links users click. Use `useNavigate` only when **code logic** should trigger navigation (after form submit, auth check, timer, etc.).

Must be inside Router

`useNavigate` only works inside a component that is **within** `BrowserRouter`. Calling it outside will throw an error.

Outline

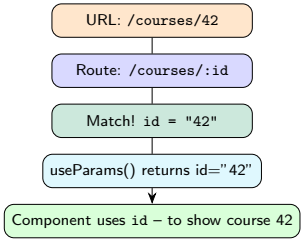
URL Parameters and Dynamic Routes

W3Schools – React Router URL Parameters

URL parameters are variables that you can add to your route paths. They are often used to **pass data between components**. React Router provides the `useParams` hook to access these parameters in your components.

```jsx
// Route definition -- :id is the parameter
<Route path="/courses/:id" element={<CourseDetail />} />

// The same component handles ANY id:
// /courses/1  --> id = "1"
// /courses/42 --> id = "42"
// /courses/swe381 --> id = "swe381"
```

```jsx
import { useParams } from 'react-router-dom';

function CourseDetail() {
  // useParams returns an object of all params
  const { id } = useParams();

  return (
    <div>
      <h1>Course: {id}</h1>
      <p>Showing details for course {id}</p>
    </div>
  );
}
```



useParams – W3Schools Customer Example

W3Schools

In the path `http://localhost:5173/customer/Tobias`, the URL parameter is **Tobias**. URL parameters let you create dynamic routes where part of the URL can change.

```jsx
import { Routes, Route, useParams, Link }
  from 'react-router-dom';

// Route definition
<Route path="/customer/:customerName"
       element={<Customer />} />

// Component reads the parameter
function Customer() {
  const { customerName } = useParams();

  return (
    <div>
      <h1>Hello, {customerName}!</h1>
    </div>
  );
}

// Links that create different URLs
function App() {
  return (
    <nav>
      <Link to="/customer/Tobias">Tobias</Link>
      <Link to="/customer/Ahmed">Ahmed</Link>
      <Link to="/customer/Sara">Sara</Link>
    </nav>
```

URL: /customer/Tobias

Hello, Tobias!

URL: /customer/Ahmed

Hello, Ahmed!

URL: /customer/Sara

Hello, Sara!

Multiple parameters

Use multiple : segments in one path:
`/course/:courseId/lesson/:lessonId`
Both are returned by `useParams()`.

useParams – Fetching Data by ID

Setup & data fetching:

```
import { useParams, useNavigate }
  from 'react-router-dom';
import { useState, useEffect } from 'react';
function CourseDetail() {
  const { id }      = useParams();
  const navigate    = useNavigate();
  const [course,  setCourse]  = useState(null);
  const [loading, setLoading] = useState(true);
  const [error,   setError]   = useState(null);
  useEffect(() => {
    const load = async () => {
      try {
        const res = await fetch(
          `...typicode.com/posts/${id}`
        );
        if (!res.ok)
          throw Error('Not found');
        setCourse(await res.json());
      } catch(e){setError(e.message);}
      finally  {setLoading(false);}
    };
    load();
  }, [id]); // re-run when id changes
```

Render (after fetch):

```
  if (loading)
    return <p>Loading {id}...</p>;
  if (error)
    return <p>Error: {error}</p>;

  return (
    <div>
      <h1>{course.title}</h1>
      <p>{course.body}</p>
      <button onClick={()=>navigate(-1)}>
        Back
      </button>
    </div>
  );
}
```

Re-fetch on id change

`id` in the `useEffect` deps ensures data re-loads when the URL parameter changes (e.g. `/courses/3` → `/courses/5`).

Outline

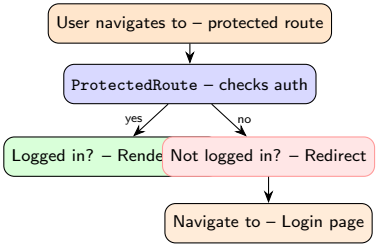
Why Protected Routes?

Problem: Some pages should only be visible to authenticated users.

- `/dashboard` – logged-in users only
- `/profile` – logged-in users only
- `/admin` – admin role only
- `/login`, `/` – public (everyone)

ProtectedRoute pattern

A **ProtectedRoute** component wraps any route that requires authentication. It checks auth status and either renders the children or redirects to `/login` using the `Navigate` component.



ProtectedRoute – Implementation

```jsx
import { Navigate } from 'react-router-dom';
import { useAuth } from './context/AuthContext';
function ProtectedRoute({ children }) {
  const { isAuthenticated } = useAuth();
  if (!isAuthenticated) {
    return <Navigate to="/login" replace />;
  }
  return children;
}
export default ProtectedRoute;
```

Using ProtectedRoute in App.jsx:

```jsx
<Routes>
  <Route path="/"
    element={<Home />} />
  <Route path="/login"
    element={<LoginPage />} />
  <Route path="/dashboard"
    element={
      <ProtectedRoute>
        <Dashboard />
      </ProtectedRoute>} />
  <Route path="/profile"
    element={
      <ProtectedRoute>
        <Profile />
      </ProtectedRoute>} />
  <Route path="*"
    element={<NotFound />} />
</Routes>
```

Not logged in – /dashboard

Redirected to /login

Email: [ ]

Password: [ ]

[Log In]

Logged in – /dashboard

Welcome to Dashboard!

Hello, Ahmed Al-Farsi!

SWE 381 – Web App Dev

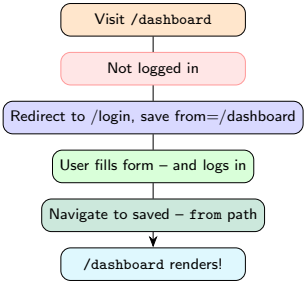
[Log Out]

replace prop

Pass `replace` to `Navigate` so that after login the user can navigate **forward** but the unauthenticated URL is removed from history.

ProtectedRoute – Redirect Back After Login

```jsx
import { Navigate, useLocation }
  from 'react-router-dom';
function ProtectedRoute({ children }) {
  const { isAuthenticated } = useAuth();
  const location = useLocation();
  if (!isAuthenticated) {
    return (
      <Navigate to="/login"
        state={{ from: location }}
        replace />
    );
  }
  return children;
}
// LoginPage.jsx -- redirect back after login
function LoginPage() {
  const navigate = useNavigate();
  const location = useLocation();
  const { login } = useAuth();
  const handleLogin = (userData) => {
    login(userData);
    const dest =
      location.state?.from?.pathname
      || '/dashboard';
    navigate(dest, { replace: true });
  };
  // ...
}
```



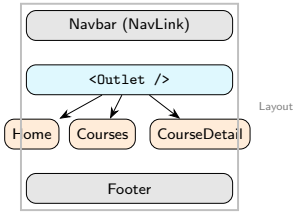
Outline

Nested Routes and Layout Component

React Router – Nested Routes

Nested routes allow a parent route to render a layout (navbar, sidebar) while child routes render the main content. The `Outlet` component marks where child routes appear.

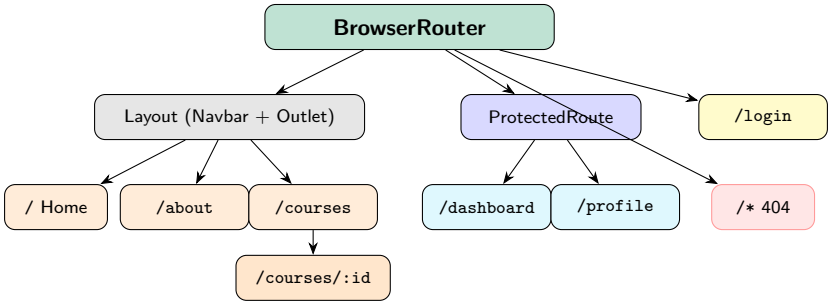
```jsx
import { Outlet, NavLink }
  from 'react-router-dom';
function Layout() {
  return (<>
    <nav>
      <NavLink to="/">Home</NavLink>
      <NavLink to="/courses">Courses</NavLink>
      <NavLink to="/about">About</NavLink>
    </nav>
    <main><Outlet /></main>
    <footer>SWE 381 -- KSU</footer>
  </>);
}
// Nested route setup in App.jsx:
<Routes>
  <Route path="/" element={<Layout />}>
    <Route index
      element={<Home />} />
    <Route path="courses"
      element={<Courses />} />
    <Route path="courses/:id"
      element={<CourseDetail />} />
    <Route path="about"
      element={<About />} />
  </Route>
  <Route path="/login"
```



index route

`<Route index ...>` is the **default child** when the parent path is matched exactly with no further path segment.

Full SWE381 Project Route Map



Full Project – Complete App.jsx

Imports:

```
import { Routes, Route }
  from 'react-router-dom';
import Layout
  from './components/Layout';
import ProtectedRoute
  from './components/ProtectedRoute';
import Home          from './pages/Home';
import About         from './pages/About';
import Courses       from './pages/Courses';
import CourseDetail  from './pages/CourseDetail';
import Dashboard     from './pages/Dashboard';
import Profile       from './pages/Profile';
import Login         from './pages/Login';
import NotFound      from './pages/NotFound';
```

App component:

```
function App() {
  return (
    <Routes>
      {/* Public pages in Layout */}
      <Route path="/" element={<Layout />}>
        <Route index element={<Home />} />
        <Route path="about"
               element={<About />} />
        <Route path="courses"
               element={<Courses />} />
        <Route path="courses/:id"
               element={<CourseDetail />} />
      </Route>
      {/* Protected -- redirect if not logged in */}
      <Route path="/dashboard" element={
        <ProtectedRoute>
          <Dashboard />
        </ProtectedRoute>} />
      <Route path="/profile" element={
        <ProtectedRoute>
          <Profile />
        </ProtectedRoute>} />
      {/* Auth page -- no Layout needed */}
      <Route path="/login" element={<Login />} />
      {/* 404 catch-all */}
      <Route path="*" element={<NotFound />} />
    </Routes>
  );
}
```

Layout Component – Navbar with NavLink

Imports & style helper:

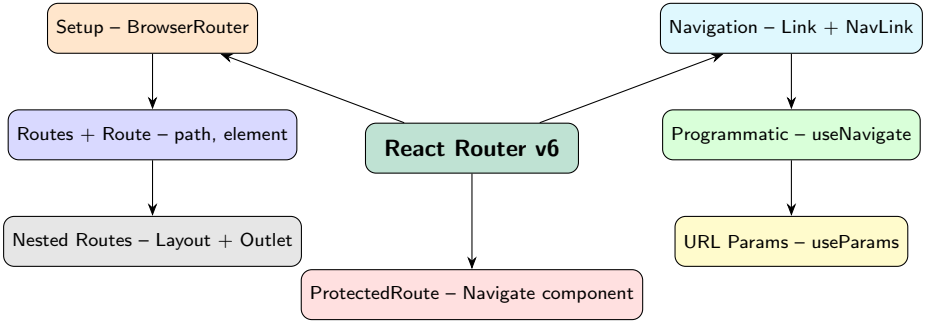
```
import { NavLink, Outlet }
  from 'react-router-dom';
import { useAuth }
  from '../context/AuthContext';

const activeStyle = ({ isActive }) => ({
  color:
    isActive ? '#007bff' : '#333',
  fontWeight:
    isActive ? 'bold' : 'normal',
  padding: '8px 16px',
  textDecoration:
    isActive ? 'none' : 'underline'
});
```

Layout component:

```
function Layout() {
  const { isAuthenticated, user, logout }
    = useAuth();
  return (<>
    <nav style={{background:'#f8f9fa',
                 padding:'10px'}}>
      <NavLink to="/" style={activeStyle}>
        Home</NavLink>
      <NavLink to="/about"
        style={activeStyle}>About</NavLink>
      <NavLink to="/courses"
        style={activeStyle}>Courses</NavLink>
      {isAuthenticated ? (<>
        <NavLink to="/dashboard"
          style={activeStyle}>Dashboard</NavLink>
        <span>Hi, {user?.name}!</span>
        <button onClick={logout}>
          Log Out</button>
      </>) : (
        <NavLink to="/login"
          style={activeStyle}>Log In</NavLink>
      )}
    </nav>
    <main style={{padding:'20px'}}>
      <Outlet />
    </main>
    <footer style={{textAlign:'center',
                    padding:'10px'}}>
      SWE 381 -- Web App Dev -- KSU
    </footer>
```

Outline

React Router – Concept Map



Lecture 9 – Key Takeaways

1. **Client-side routing** – React Router intercepts navigation, swaps components without page reload, and preserves state
2. **Three core pieces**: `BrowserRouter` (wraps app) → `Routes` (container) → `Route` (path + element pairs)
3. `Link` prevents page reloads; `NavLink` adds active styling via the `isActive` prop in its `style`/`className` function
4. `useNavigate` – returns a function for programmatic navigation; used after form submit, auth check, or timers
5. **URL parameters** (`:param` in path) create dynamic routes; `useParams` reads them as an object
6. `useParams` values are always **strings** – convert with `Number()` when needed; include the param in `useEffect` deps
7. **ProtectedRoute** wraps private pages; renders `<Navigate to="/login">` when user is not authenticated
8. **Nested routes** with `<Outlet />` allow a **Layout** component (navbar, footer) to persist across page changes
9. **Route map**: public routes in Layout, protected routes wrapped with ProtectedRoute, `path="*"` as 404 catch-all

Course Complete: React Fundamentals (L1–L9) → Ready to build full-stack SPAs

Practice Exercises

1. Install `react-router-dom` in a Vite project. Create three page components: **Home**, **About**, and **Contact**. Wrap the app with `BrowserRouter` and set up three `Route` entries in `App.jsx`.
2. Replace plain `<a>` tags with `Link` components. Confirm that navigating between pages does not reload the browser. Check the Network tab in DevTools – no HTML requests should appear after the first load.
3. Replace `Link` in your navbar with `NavLink`. Add a style function that makes the active link bold and blue. Test by clicking each menu item and confirming only the current page's link is highlighted.
4. Create a `/courses` page listing 5 courses. Each should be a `Link` to `/courses/:id`. Build a `CourseDetail` page that reads the `id` with `useParams` and displays it.
5. Add `useNavigate` to a Login form. On submit, navigate to `/dashboard`. Add a "Back" button on the dashboard that calls `navigate(-1)`.
6. Build a **ProtectedRoute** component using the `AuthContext` from Lecture 8. Wrap `/dashboard` and `/profile` with it. Test: confirm visiting those URLs while logged out redirects to `/login`.
7. **Challenge**: Implement the full SWE381 project route map – `Layout` with `Outlet` for the persistent navbar, nested public routes, ProtectedRoute for dashboard and profile, and a `path="*"` 404 page. Confirm the navbar stays visible when navigating between pages.